

Islamic University of Technology

Course Code : EEE 4308
Course Name : Digital Electronics Lab
Project Name : SAP-1 Architecture-based 8-bit Computer Design and Implementation.
Phase No. : A

Team : Byte_Masters

Name	Student ID
Irfan Hossain Rafi	230021331
Rafin Islam	230021323
Sanajid Arham Bakar	230021311
Raad al Hasan	230021322
Nazifa Noushin	230021117
Samiha Alam Purna	230021127

Supervisor:
Mr. Md. Rahib-Bin-Hossain

Junior Lecturer, Electrical & Electronic Engineering Department

Islamic University of Technology

Project Objective:

Objective: The project's goal is to create an 8-bit computer following the SAP-1 (Simple-As-Possible) architecture by utilizing a combination of combinational and sequential circuits.

Brief Description of Project: The computer project aims to incorporate several key features and functionalities within its design. These include the utilization of a single 8-bit bus for both address and data transfer, ensuring efficient communication within the system. An 16x8 bit random access memory (RAM) is integrated for the storage of programs and data, enabling data manipulation and program execution. To manage program execution, the computer incorporates a 4-bit program counter that can effectively cycle through addresses from 0000 to 1111. Furthermore, an input unit is included to facilitate the initial writing of programs into the RAM, enabling the computer to run specific tasks. The memory address register (MAR) plays a crucial role in accessing data stored in the RAM.

For arithmetic and logic operations, the computer is equipped with an arithmetic and logic unit (ALU) capable of 8-bit addition and subtraction. It employs two registers, the Accumulator (register A) and register B, each with a capacity of 8 bits, for storing data necessary for ALU operations. To ensure the coordinated functioning of all components, a dedicated control unit is integrated, overseeing and synchronizing the activities of registers, memory, ALU, and other vital elements. Finally, the computer has an output unit that can display results in binary form, allowing users to interpret the output of their computational tasks effectively. These combined features and functionalities provide a comprehensive foundation for the successful operation of the 8-bit computer based on the SAP-1 architecture. It has the ability to perform the following instruction set-

Operation	Description
Load	Load data from memory to the accumulator
Add	Add data from memory to value in the accumulator
Subtract	Subtract data from memory from value in the accumulator
Output	Load data from accumulator to the output register
Halt	Stop processing

The Main clock signal driving the computer should have two modes of operation- (i) Auto and (ii) Manual. Our SAP -1 must have all the blocks shown in the project diagram. Specifications of each block are described later.

Submodules:

Clock Pulse Generator

The Main Clock serves as the primary heartbeat of the computer, providing the driving signal necessary to synchronize all internal components. To allow for both high-speed execution and careful debugging, the system is designed with two distinct operational modes: Auto and Manual.

At its core, the clock signal is generated using the versatile 555 timer IC. By configuring this integrated circuit into different types of multivibrators, we can produce a steady, automatic pulse (Astable mode) or a single, user-triggered pulse for manual stepping (Monostable mode).

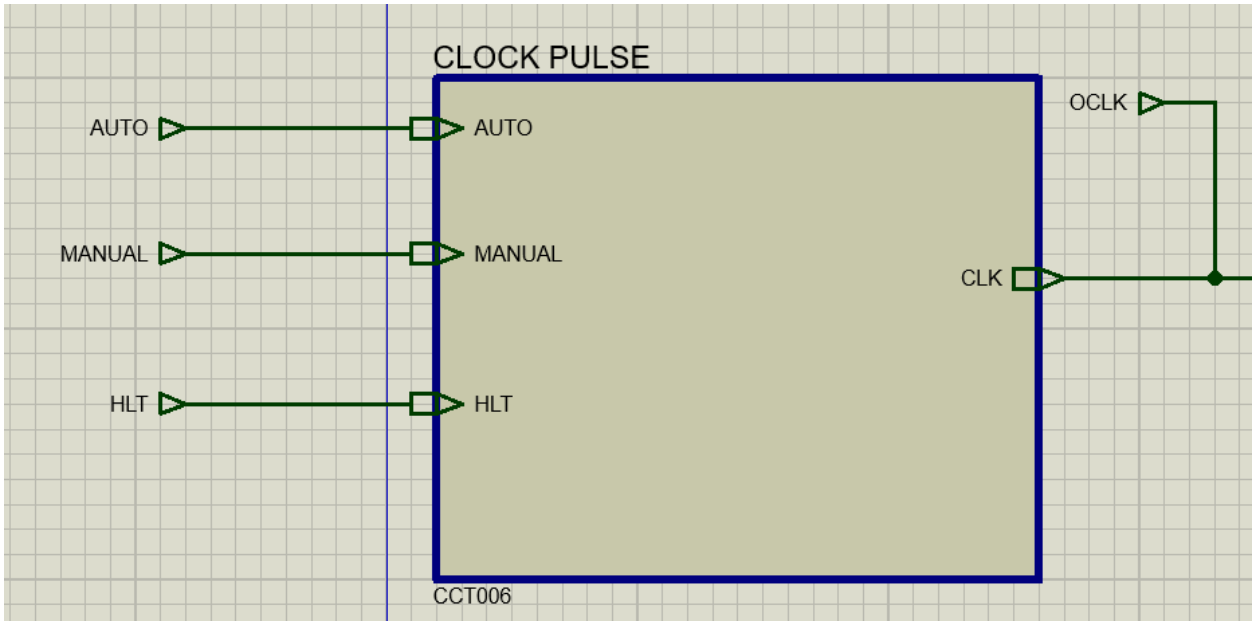
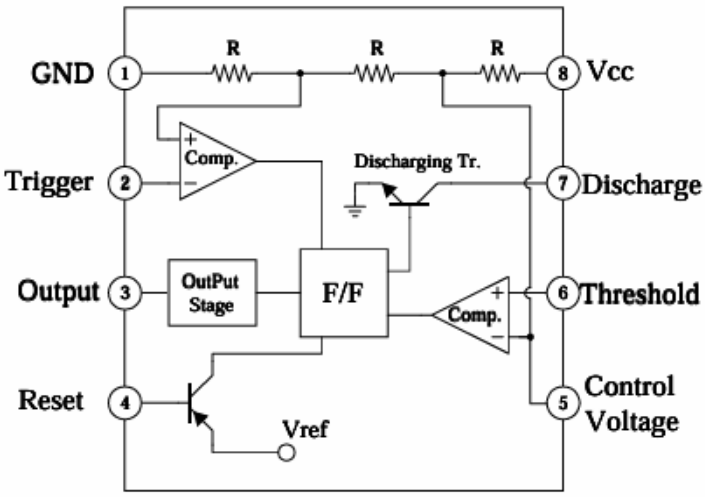


Fig: Block diagram of Clock Pulse

IC Components Used:

Based on the circuit design, the following components and integrated circuits are utilized to construct the clock module:

- **Integrated Circuits (ICs):**
    - 555 Timer (x2): Labeled U6 and U7. U6 is configured for astable operation (Auto), and U7 is configured for monostable operation (Manual).



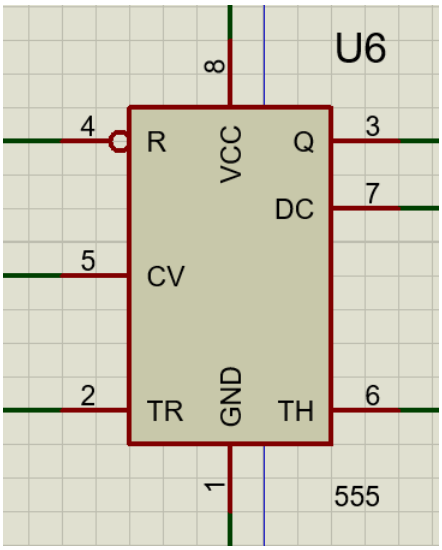


Fig: Schematic of 555 timer IC

Fig: Block diagram of 555 timer IC in Proteus

- 7404 Hex Inverter: Labeled U5 and U8. Used for signal inversion in the selection logic.

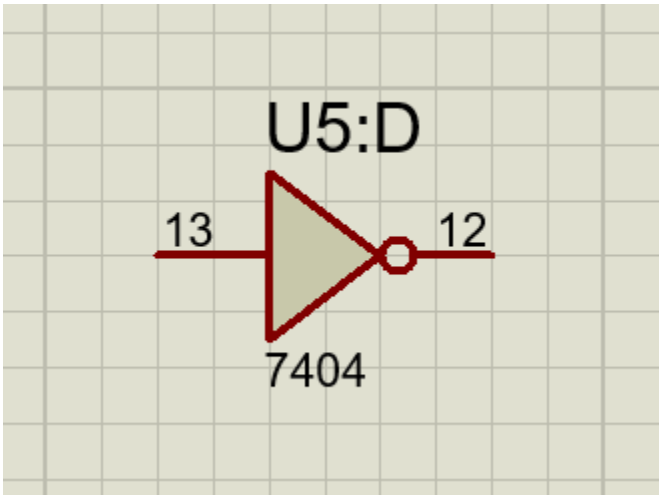


Fig: 7404 Hex inverter

- 7408 Quad 2-Input AND Gate: Labeled U9. Used to gate the signals and implement the final Halt logic.

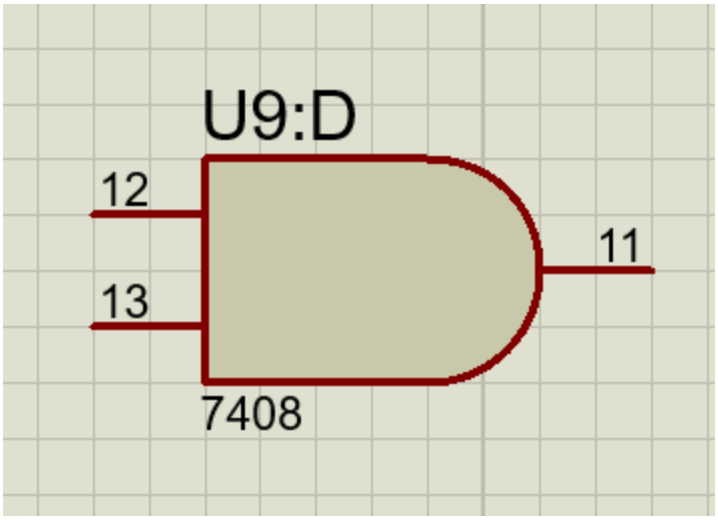


Fig: 7408 AND Gate

- 7432 Quad 2-Input OR Gate: Labeled U10. Used to combine the Auto and Manual signals.

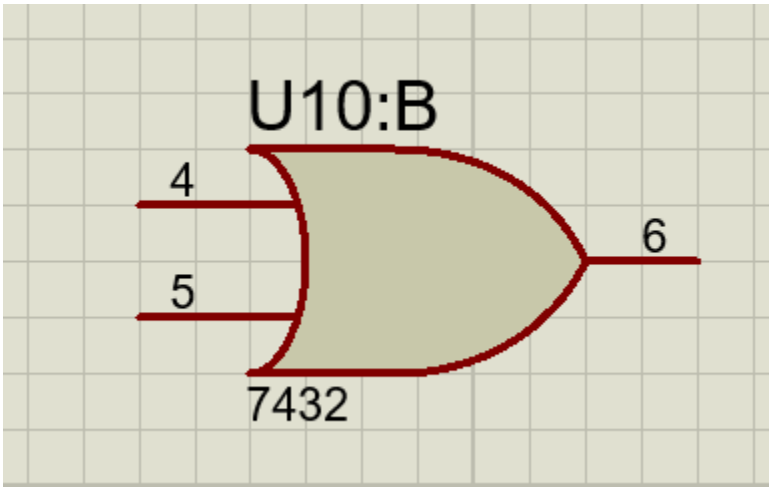


Fig: 7432 OR Gate

- **Passive Components:**
  - Resistors:
    - 100k ohms (R1) and 10k ohms (R2) for Astable timing.
    - 1M ohms (R4) for Monostable timing.
    - 220 ohms (R3, R7) for LED current limiting.
    - 10k ohms (R6) and 1k (R5) for pull-up/pull-down resistors.

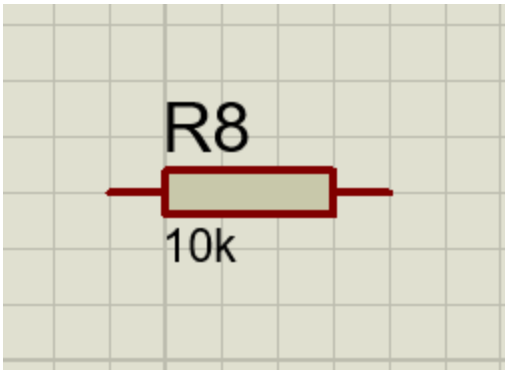


Fig: Figure of Resistor

- Capacitors:
  - 1 micro farad (C1) for Astable frequency.
  - 0.1 micro farad (C4) for Monostable pulse width.
  - 0.01 micro farad (C2, C3) for noise filtering on the Control Voltage pin.

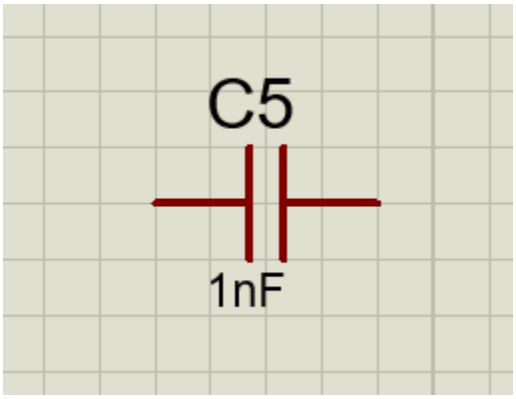


Fig: Figure of Capacitor

- **I/O Devices:**
  - LEDs: Green (D1) to indicate Astable output; Pink (D2) to indicate Monostable output. It is mainly for debugging purposes.

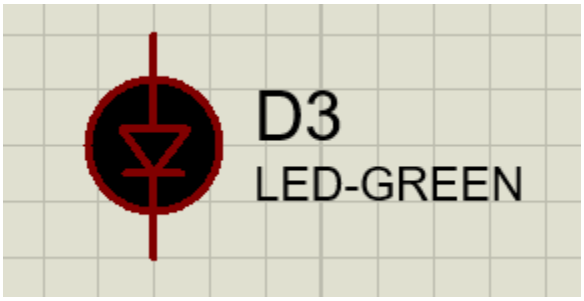


Fig: Figure of LED

- Logic Inputs: Connection points for AUTO selection, MANUAL trigger, and HLT (Halt) signal.

Description of Connection:

The clock module circuit is constructed by combining two independent timing circuits which are then fed into a control logic block.

A. Astable Multivibrator (Auto Mode)

- Timer U6 is configured in astable mode to generate a continuous pulse.
- Pins 2 (Trigger) and 6 (Threshold) are shorted together and connected between the timing capacitor C1 and the resistor network R1 and R2 .
- Discharge Pin 7 is connected between R1 and R2.
- The output from Pin 3 connects to the Green LED (D1) and feeds into the first logic gate U9:A.

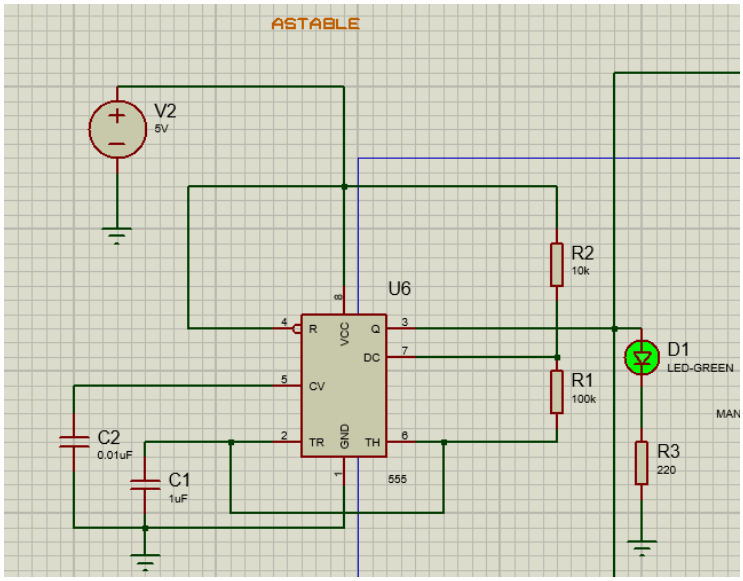


Fig: Astable Multivibrator

B. Monostable Multivibrator (Manual Mode)

- Timer U7 is configured in monostable mode to generate single pulses.
- The Trigger Pin 2 is held high by a pull-up resistor (R5) and connected to a push-button via the MANUAL input.
- The timing network consists of resistor R4 and capacitor C4 connected to the Threshold and Discharge pins.
- The output from Pin 3 connects to the Pink LED (D2) and feeds into the second logic gate U9:B.

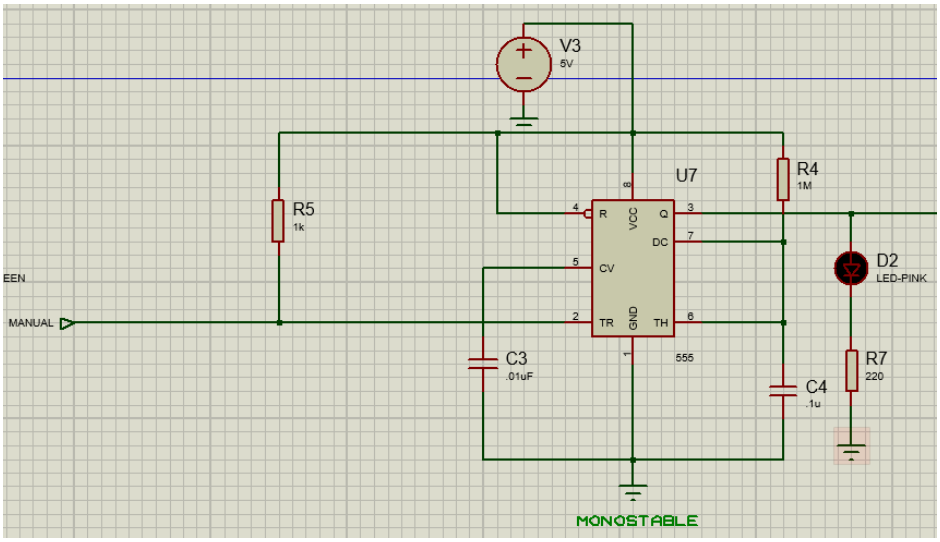


Fig: Monostable Multivibrator

C. Control & Halt Logic

- The AUTO selection signal is inverted by U5:C and U8:B to toggle between the two timer outputs using AND gates (U9:A and U9:B).
- The selected signal passes through OR gate U10:A.
- Finally, the signal is gated by U9:C, which is controlled by the HLT (Halt) input, ensuring the clock stops when required.

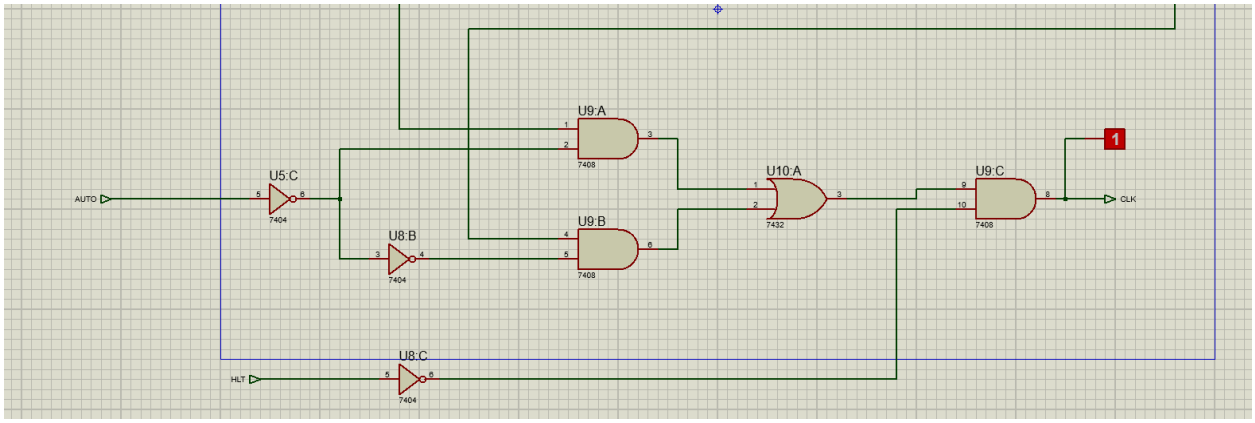


Fig: Control and Halt Logic

Overall Schematic Diagram:

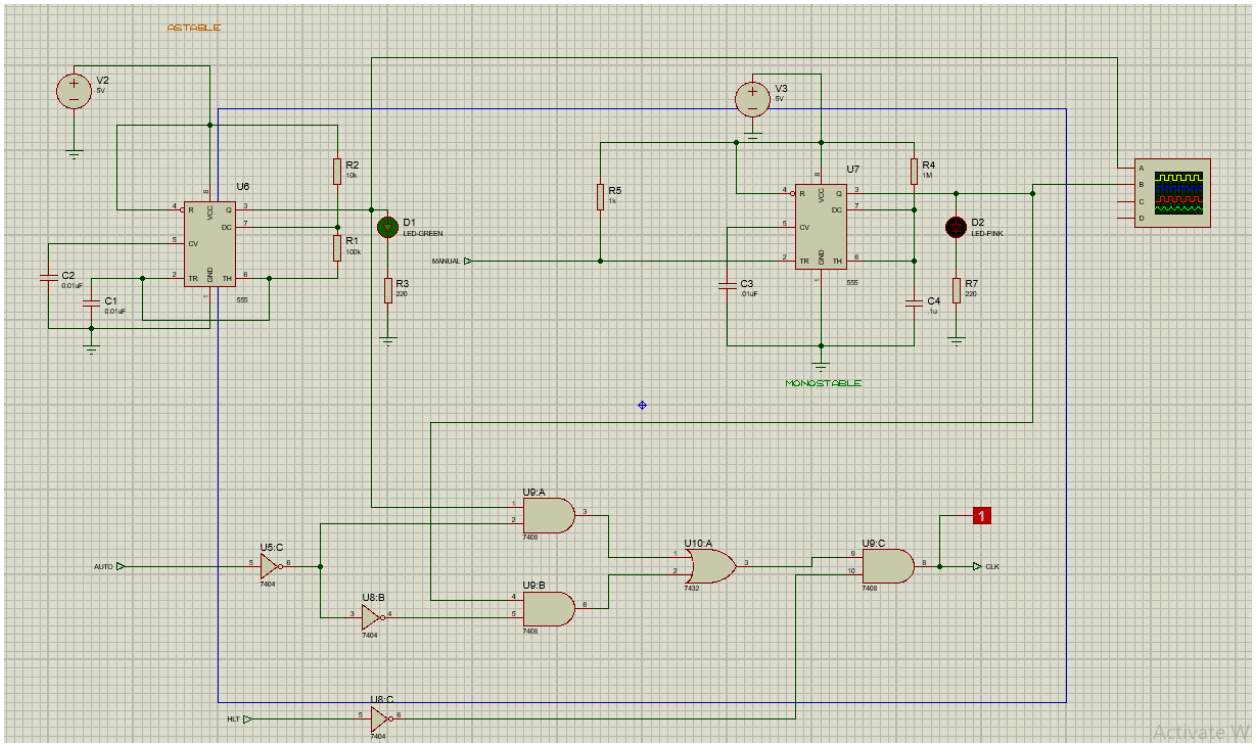


Fig: Overall Diagram of Clock Pulse Generator on Child Sheet

Functions:

This module performs several critical roles within the 8-bit computer architecture:

- **Dual-Mode Clock Generation:** It generates two distinct types of timing signals:
    1. **Auto Mode:** A continuous square wave frequency for normal program execution.
    2. **Manual Mode:** A single-step pulse for debugging and instructional analysis.
- **Signal Multiplexing:** It utilizes logic gates to select between the Astable and Monostable sources without signal collision, controlled by a simple user toggle.
- **System Halt:** It integrates a "Halt" priority input that can override all other signals to freeze the computer state instantly.
- **Visual Monitoring:** It provides real-time status indication via specific LEDs (Green for Auto, Pink for Manual) to verify signal activity.

Working Principle:

The circuit operation is best understood by analyzing its three distinct stages:

i. Astable Multivibrator (Auto Mode)

The Auto clock signal is generated by timer U6, configured in astable mode. In this configuration, the timing capacitor C1 charges through resistors R1 and R2 until the voltage reaches 2/3 Vcc. The internal comparator then triggers the discharge pin (Pin 7), draining the capacitor through R2 until it drops to 1/3 Vcc. This continuous charge/discharge cycle creates a steady oscillating square wave output at Pin 3.

ii. Monostable Multivibrator (Manual Mode)

The Manual clock signal is generated by timer U7, configured as a monostable multivibrator (one-shot). The circuit remains in a stable Low state until the Manual button is pressed, which grounds the Trigger input (Pin 2). This sets the internal flip-flop to High. The output remains High only for the duration it takes capacitor C4 to charge through resistor R4 to 2/3 Vcc (T = 1.1 x R4 x C4). This ensures exactly one clean pulse is generated per button press, eliminating switch bounce issues.

iii. Logic Control & Gating The final output is determined by a logic network comprising Inverters (7404), AND gates (7408), and OR gates (7432).

- **Selection:** The **AUTO** input drives an inverter logic that alternately enables one of two AND gates. This allows only the selected signal (Astable or Monostable) to pass through to the OR gate, effectively acting as a multiplexer.
- **Halt Priority:** The final signal passes through a specific AND gate controlled by the **HLT** (Halt) input. If the HLT signal is active (High), the inverter forces the AND gate input Low, permanently blocking the clock signal and freezing the CPU operations regardless of the mode selected.

Duty Cycle:

To verify the stability of the clock signal in Auto Mode (Astable), we analyzed the output waveform using the oscilloscope graph. The duty cycle represents the percentage of the total period during which the signal is active (High). Ideally, a square wave for a clock signal should be close to 50% to ensure balanced timing for the computer's logic operations.

Based on the measured values from the simulation graph:

- Pulse Width (High Time): 0.765 ms
- Total Time Period: 1.47 ms

Calculation: Duty Cycle (D)

= (Pulse Width / Total Time Period) * 100%

= (0.765 ms / 1.47 ms) * 100%

= 52.04%

Result Analysis: The calculated duty cycle is approximately 52%. This indicates that the clock signal is High for slightly more than half of the cycle. This is an acceptable range for the SAP-1 architecture, ensuring that the components have sufficient time to trigger and settle during the active state. Moreover, we used the cursor to identify the points, so there might be some slight deviation from the actual value.

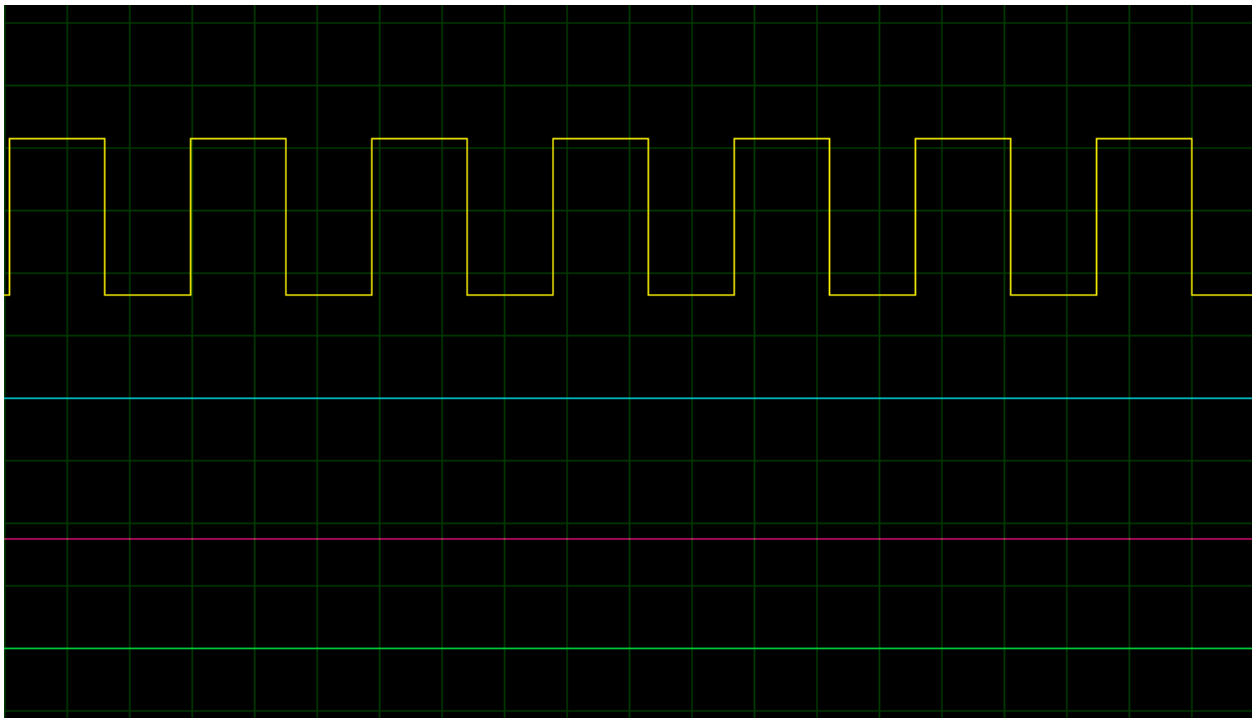


Fig: Oscilloscope reading

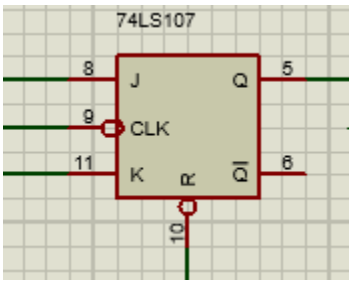
Program Counter

A Program Counter is a 4-bit counter that can count from 0000 to 1111 (In terms of decimal number system, 0 to 15) and we will be using this in our SAP-1 (8-bit) It keeps track of data by storing and sending memory address to the RAM to fetch the next instruction. It then executes them one by one and then for each execution or in other words, instruction cycle, its value will increase by 0001. This increment allows the Program Counter to get to the next instruction in RAM, and then execute them as before. Like other modules, Program Counter is also connected to the Bus to fetch or send its data.

IC/Components used:

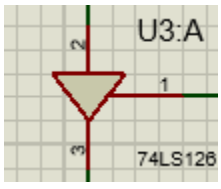
In this EEE 4308 project, we designed the modules for SAP-1 in Proteus 8 Professional (Version 8.12), including the Program Counter. If we take a look at the child sheet of our Program Counter, we have made use of various components in the software's library to build it.

74LS107 (J-K Flip-Flops):



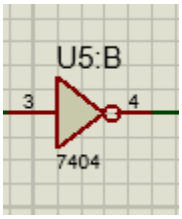
These are used to store the binary count in the program counter. They help keep track of the current value and generate the clock signals required for the counting process, and therefore ensures that the program counter is updated correctly with each clock pulse.

74LS126 (Buffers):

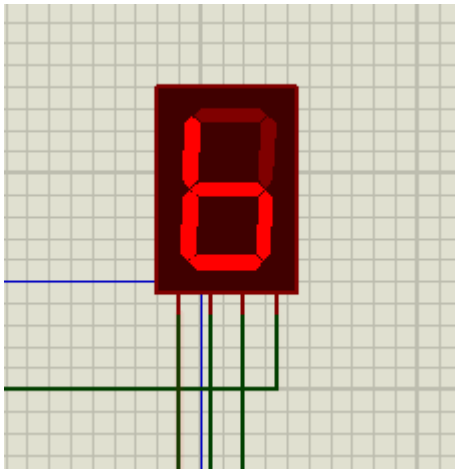


They allow some modules to be deactivated or activated while being connected to the Bus. In the Program Counter we used 74LS126

7404 (Inverters): Used to invert the signals from high to low (and vice versa), helping make sure the flip-flops receive the correct timing and change state accordingly.



7-Segment Display: This component displays the count number coming from the program counter to make it easy to read.



Connections of the Parent Sheet:

Input Pins:

CLK (Clock):
The clock pin is connected to the clock pulse generator. Every time a clock pulse occurs, the program counter increases its value by one. This allows the system to move through the instruction addresses.

CE (Count Enable):
The count enable pin decides whether the program counter should count or not. When CE is on, the counter responds to the clock and increments normally. When CE is off, the counter pauses and keeps its current value, even if clock pulses are present.

CLR (Clear):
The clear pin resets the program counter back to 0000, which is often at the beginning so that the program counter always starts executing from the first memory address.

CO (Carry Out):
The carry out pin becomes active when the counter reaches its maximum value and rolls over from 1111 back to 0000. This signal can be useful for extending the counter or for control purposes, although it is not used directly in the display.

Output Pins

Q0, Q1, Q2, Q3:
These pins give the 4-bit binary output of the program counter. Q0 is the least significant bit and Q3 is the most significant bit . Together, these outputs represent the current value stored in the program counter.

Display Connection

7-Segment Display:
The outputs from Q0 to Q3 are connected to the 7-segment display circuit. This allows the binary value of the program counter to be converted into a visible number, making it easy to monitor the current count while the system is operating.

Connections inside the Child Sheet of Program Counter:

Clock (CLK):
The external clock signal first passes through a 7404 inverter. This inverted clock is then connected to the clock inputs of all JK flip-flops. Inverting the clock ensures correct triggering and proper timing of the flip-flops.

Count Enable (CE):
The CE signal is connected to the J and K inputs of all JK flip-flops. When CE is high, both J and K are enabled, allowing the flip-flops to toggle and the counter to increment on each clock pulse. When CE is low, the flip-flops hold their current states.

Clear (CLR):
The clear signal is inverted using a 7404 inverter and then connected to the asynchronous reset inputs of all flip-flops. Activating CLR resets all flip-flops to zero, setting the program counter value to 0000.

JK Flip-Flop Chain (74LS107)

The program counter is built using four 74LS107 JK flip-flops, each representing one bit of the 4-bit counter since we are using a 4-bit Program Counter. The Q output of each flip-flop provides one bit of the counter value.
Each successive flip-flop is driven by the output of the previous stage, forming a ripple counter structure that increments in binary sequence.

Outputs

The outputs of the flip-flops (Q0, Q1, Q2, Q3) are connected to 74LS126 tri-state buffers. These buffers control when the program counter places its value onto the system bus. When the buffer enable signal is active, the counter outputs are passed to the bus. When disabled, the outputs remain in a high-impedance state, preventing bus conflicts.

Carry Out (CO) Signal

The CO (Carry Out) is an input pin, sometimes called Program Counter Out. When it's high, it allows data to flow from the Program Counter to the BUS of the SAP-1. When it's low, the data transfer stops. We keep it on while the system is running so the Program Counter has an output and can send data to other parts of the SAP-1 as needed and in this phase, 7-segment display.

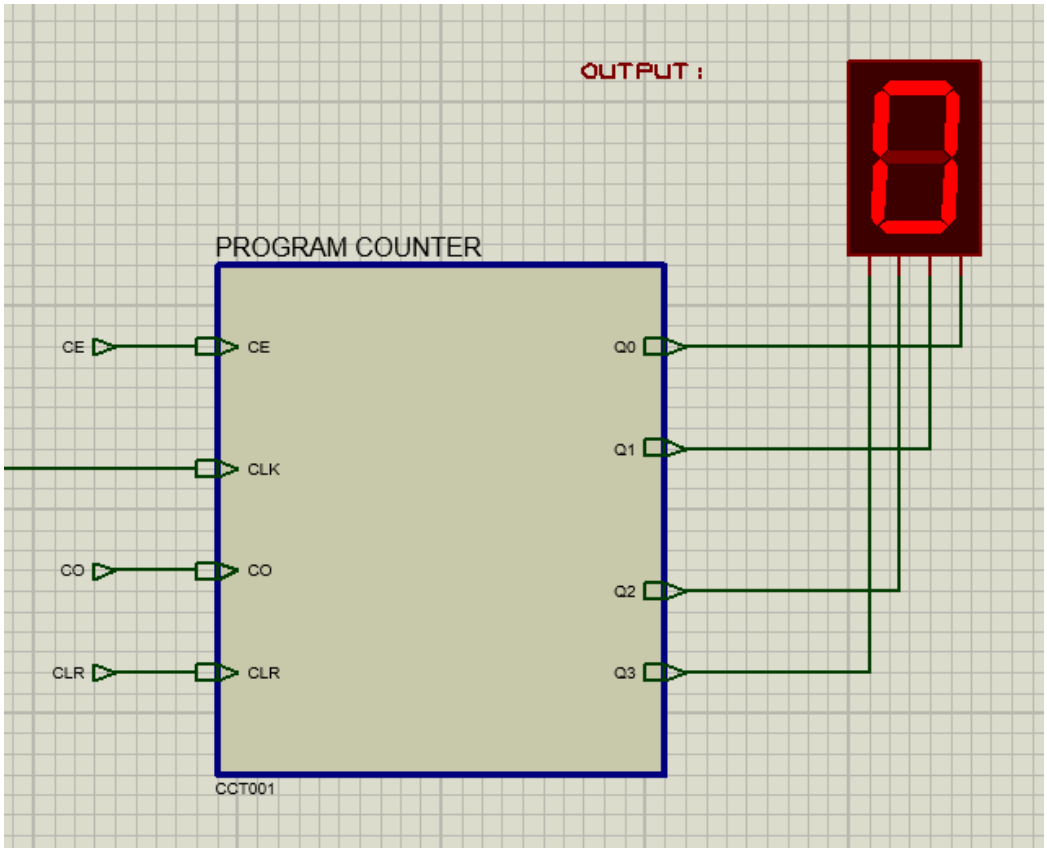


Figure 1:- Parent Sheet of Program Counter

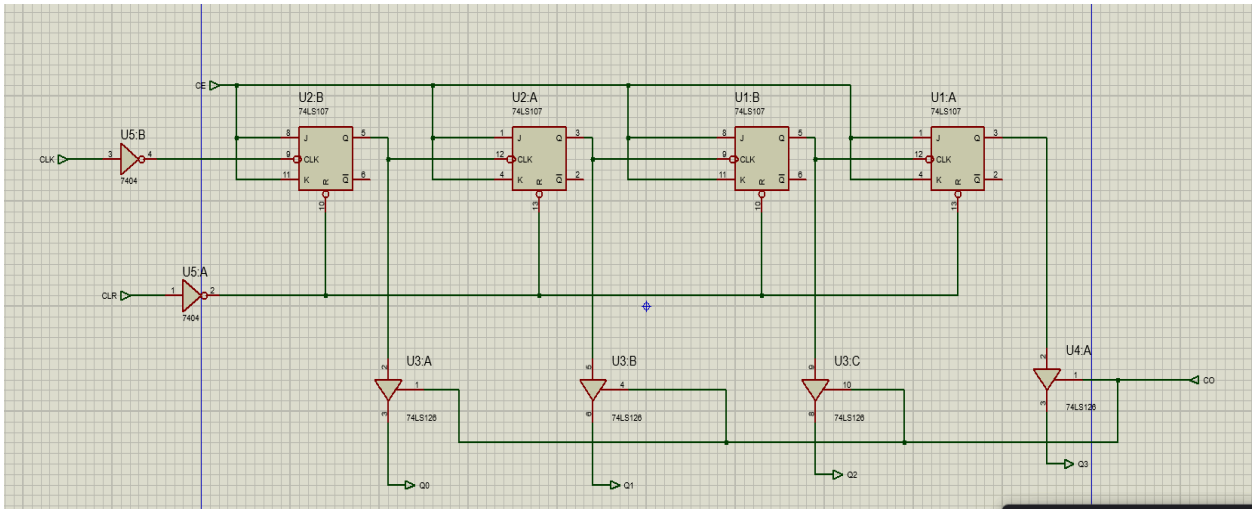


Figure 2:- Child Sheet of Program Counter

Functions:

The program counter is an essential part of the SAP-1 architecture that keeps track of the address of the next instruction to be executed.

When the count enable signal is active, the program counter increments by one on each clock pulse, allowing instructions to be fetched and executed in the correct sequence. It stores the current instruction address as a 4-bit binary value, which is used to access memory.

The clear (CLR) signal resets the program counter to 0000, ensuring that program execution always starts from the first memory location.

The count enable input also allows the PC to hold its value when required, making it possible to pause instruction sequencing.

Additionally, the Program Counter places its output onto the system bus through tri-state buffers, allowing other components to read the address without causing bus conflicts, while the carry out (CO) signal is used to turn the output of Program Counter on/off.

Overall, the program counter ensures smooth and orderly program execution by reliably generating, updating, and supplying instruction addresses throughout the operation of the system.

Working Principle:

When the Clock (CLK) signal pulses, the Program Counter (PC) will increment by one, updating its stored value, which is shown on the 7-segment display. So, let’s say the PC starts at 0000. When the Clock (CLK) pulse comes in, each flip-flop updates its state, incrementing the counter by one; turning 0000 to 0001. However, because of the tiny delays (in the nanosecond range) inside each flip-flop, the updates don’t happen instantly; they take a little bit of time for the signal to propagate through the flip-flops.

When we send a Clear (CLR) signal, the counter will reset back to 0000, clearing out any stored value. Now, if the counter hits 1111 (which is 15 in decimal), the next clock pulse will make it back to 0000 due to an overflow, which will be indicated by Carry-Out (CO) signal signaling that the counter has overflowed. The Program Counter then in the next cycle fetches the next instruction one step at a time, following the clock pulses and adjusting to the system's timing.

Discussion:

In Phase A of our project, we focused on setting, building and designing the Program Counter and Clock Pulse for the 8-bit SAP computer. The Clock module was created to provide synchronization signals in both Auto and Manual modes with the help of Monostable and Astable Multivibrator, while the Program Counter was designed to generate memory addresses sequentially. By successfully integrating these components we have set up the foundation for reliably fetching and executing instructions in the later stages of this project. Initially our pulse was not stable and counting was fast due to fast charging and discharging of our capacitors. In order to smoothen the pulse and slow down the counting, we reduced the value of our capacitors and hence we obtained a smoother pulse. Also another reason we obtained a smoother pulse was due to the fact that the duty cycle we obtained is around 52.04% which is approximately close to 50%.

Through this phase, we developed a deeper understanding of 555 IC timer Astable and Monostable multivibrator, and made use of it to build a clock pulse and we also learned that for auto mode, Astable multivibrator is used and for manual mode, monostable multivibrator is used. We use a monostable multivibrator instead of a regular switch to implement a debouncing circuit. Also in this phase, we learned the use of Program Counter, how to build and design it as well as its working principle. Overall, this phase provided significant practical insights into the intersection of analog timing and digital sequential logic.

Credits:

Work Contribution	Contributors
Clock Pulse Generator	Samiha (Astable, Monostable) and Rafi (rest of the parts)
Program Counter	Nazifa and Raad
Report writing	Rafin and Sanajid